

Relations, from Zero

*What a type relation is, which ones Spacetime needs,
and what a maximalist relational semantics would buy*

Spacetime architecture note · 2026-08-13

Part I — What a relation is

A **relation** is a yes/no question about a pair of things. That is the whole definition. Write $A \ R \ B$ for “A is related to B under R”.

You already use dozens. $3 < 5$ is a relation on numbers. $"cat" == "cat"$ is a relation on strings. $Dog \leq Animal$ is a relation on types.

What makes relations interesting is not the question but the **properties** the question happens to have. Three properties matter here, and each one is a promise about what you may conclude without checking.

Reflexive: $A \ R \ A$, always

$\leq$ on numbers is reflexive: $5 \leq 5$. Strict $<$ is not: $5 < 5$ is false.

Practical use: if your relation is reflexive, you never need to special-case “the same thing on both sides.”

Symmetric: if $A \ R \ B$ then $B \ R \ A$

$==$ is symmetric. $\leq$ is not — $3 \leq 5$ does not give $5 \leq 3$.

Practical use: symmetry halves your table. If you store $A \ R \ B$ you need not store $B \ R \ A$.

Transitive: if $A \ R \ B$ and $B \ R \ C$, then $A \ R \ C$

$\leq$ on numbers is transitive: from $2 \leq 3$ and $3 \leq 5$ you may conclude $2 \leq 5$ **without looking**. That last part is the point.

Why transitivity is the load-bearing property

Transitivity is what lets you store a few **direct** facts and derive the rest. Store $length \leq length_percentage$, store $percentage \leq length_percentage$, and a transitive relation gives you every chain through them for free.

It is also what makes a relation **dangerous** when you did not want it. We will see one relation below where transitivity would destroy the type system, and where it is therefore deliberately absent.

A relation as a graph

Every relation is a directed graph. Nodes are types; an edge $A \rightarrow B$ means $A \ R \ B$. Then:

- **reflexive** = every node has a self-loop
- **symmetric** = every edge has a twin pointing back
- **transitive** = whenever you can walk $A \rightarrow B \rightarrow C$, there is a direct $A \rightarrow C$

Computing “all the edges implied by transitivity” is called the **transitive closure**. It is a graph reachability problem — from each node, walk everything you can reach. For 14 types that is trivially fast, and it is why the type relations can be stored as a handful of DATA rows rather than a big table.

Part II — The relations Spacetime needs

1. $\leq$ — precision, or subtyping

Read $A \leq B$ as “anywhere a B is wanted, an A will do.”

```
length    ≤ length_percentage
percentage ≤ length_percentage
```

Properties: reflexive, transitive, **not** symmetric. `length ≤ length_percentage` but not the reverse — a slot wanting exactly a length must not accept `50%`.

It is a **partial** order: “partial” because most pairs are unrelated, and that is correct. `color` and `length` have no edge and never should.

What firing means: you may CLAIM. If $A \leq B$ you know the value fits and you know its type; downstream may rely on it.

2. $\sim$ — consistency (Siek and Taha, gradual typing)

Read $A \sim B$ as “these might turn out compatible; I have no grounds to refuse.”

```
Unknown ~ T      for every T
T ~ Unknown      for every T
T ~ T
```

Properties: reflexive, symmetric, and **deliberately NOT transitive**.

That last one is the trick, and it is easy to miss:

```
Unknown ~ length    ✓
Unknown ~ color     ✓
length ~ color      ✗ ← does NOT follow
```

If $\sim$ were transitive, every pair of types would be related by chaining through `Unknown`, and the type system would be decoration.

What firing means: you may ADMIT, but not claim. `--ink` in a colour slot is admitted, and nothing is concluded.

Why not just make Unknown the top of a lattice?

The instinct is: put Unknown above everything, done. It fails, and the failure is instructive.

“Top” in a lattice means **the type that could be anything**. If `Unknown` is top, then `length ≤ Unknown` and `color ≤ Unknown`, and because $\leq$ IS transitive you can chain — every type flows into every other through the top. Worse, top LICENSES CLAIMS: a value of top type is a value you may substitute anywhere.

What we need is the opposite: something admitted everywhere that licenses nothing. That is not a lattice position. It is a second relation, and it works precisely because it refuses to be transitive.

3. $\sqcup$ — join (least upper bound)

Read $A \sqcup B$ as “the most specific type that covers both.”

```
length  $\sqcup$  percentage = length_percentage
color   $\sqcup$  length      =  $\perp$  (no common type – a real error)
```

Needed when one value has two possible sources: a directive parameter with a default, an arm of a conditional, a compound where two components must agree.

Join is what upgrades a **checker** into an **inferencer**. Without it you can only verify a type someone declared. With it you can compute one nobody wrote down.

4. $\sqsubseteq$ — precision ordering on gradual types

Read $A \sqsubseteq B$ as “B is more precise than A.”

```
Unknown  $\sqsubseteq$  length      // learning it is a length is an improvement
length  $\sqsubseteq$  length      // reflexive
```

Distinct from $\leq$. $\leq$ is about **substitutability** (may I use this here?); $\sqsubseteq$ is about **knowledge** (how much do I know?).

This is what lets a value be admitted as `Unknown` early and refined later **without invalidating the earlier acceptance**. The refinement moves up $\sqsubseteq$; the admission was already justified by $\sim$.

5. Unit compatibility — a relation that must NOT be an edge

`600` means 600 milliseconds in some directives (`duration:number = 600`) and 0.05 **seconds** in others (`stagger: 0.05`). So `number` sometimes stands in for `time`.

The temptation is an edge `number  $\leq$  time`. **Do not.** $\leq$ is transitive and global; that edge would make every bare number a time everywhere, and a blanket migration once made staggers 1000× too slow across twenty files with a green build.

Instead it is a **ternary** relation — three arguments, not two:

```
accepts_unitless(directive, property, unit_policy)
```

The unit belongs to the DECLARATION SITE, not to the type.

Part III — The relations as one table

sym	reading	refl	sym	trans
$\leq$	substitutable for	yes	no	yes
$\sim$	not refusible	yes	yes	NO — on purpose
$\sqcup$	least common type	—	yes	—
$\sqsubseteq$	more precise than	yes	no	yes

And the decision procedure, which is the entire type checker:

```
given expected type E and observed type O:
  if  $0 \leq E$     → accept, and CLAIM E downstream
  else if  $0 \sim E$  → accept, claim NOTHING
  else           → diagnostic
```

Three lines. Everything else is table lookup and recursion into compounds.

Part IV — Why this fits Spacetime specifically

Most languages that need type relations also need **inference**: type variables, unification, substitution, occurs checks, principal type schemes. Hindley-Milner exists to solve “what is the type of this function I never annotated?”

Spacetime has no functions in that sense, no polymorphism, no let-generalisation. Every type is a name from a 14-row table. There is nothing to infer in the HM sense — there is only **checking a value against an expected type, where either side may be unknown**.

That is exactly the gradual-typing setting, and it is why the core is 30–60 lines instead of a subsystem. The machinery HM brings would be unused, and unused machinery in a compiler is not free: it is a thing future readers must understand before they may change anything.

The honest scope of what this buys

A relation table does not make bad values good. It makes the ANSWER consistent across every place that asks. Today 55 sites ask “is this CSS or Spacetime?” and can disagree; the relation makes disagreement structurally impossible, because there is one table and one procedure.

Part V — The maximalist reading

Take the relational view seriously and it stops being about types. Spacetime is already unusual in one specific way: **its type system is data**. `%scalar_type` rows say what a value means; `%capture_type` productions say what it looks like. Both live in the standard library, editable without touching Rust.

If the RELATIONS are also data, several existing hand-written mechanisms become rows in a table.

What becomes derivable

Coercion `%widens_to` rows on `%scalar_type` give $\leq$. A new scalar declares what it widens to, and every consumer’s acceptance updates. No Rust edit.

Overload resolution several `%form`s match the same directive; today the winner is decided by a scoring function. With $\leq$ it is the JOIN — the most specific form whose parameters accept the arguments. Specificity stops being a heuristic and becomes an order.

Diagnostics with suggestions if `0` is not accepted by `E`, the table already knows which types ARE accepted by `E`. “expected `length`, got `color`; `length_percentage` would accept both” is a query, not a hand-written hint.

Editor affordances `%scalar_type` already carries `%widget`. With $\leq$ an editor can offer every widget valid at a slot, not just the exact type’s.

Migration safety a `%migration` that retypes a parameter can be CHECKED — does the new type accept everything the old one did? That is `old ≤ new`, a table query. A widening migration is provably safe; a narrowing one lists exactly which corpus values it breaks.

The one thing to be careful about

Relational thinking invites you to add edges, and every edge is a claim about the whole language forever. The `number ≤ time` example is the cautionary tale: locally reasonable, globally catastrophic, and silent.

So the discipline is: an edge must be true **unconditionally, in every directive, in every property**. Anything conditional is a ternary relation keyed by the site, or it is not a relation at all.

Part VI — Where the boundary actually is

The grammar can express more than expected, and less than hoped. Measured:

sigil	grammar can classify?	why
<code>\$sig</code>	YES	<code>binding</code> builtin; routes to its own arm
<code>--ink</code>	YES	<code>token_ref</code> production
<code>0 -> 1</code>	YES	<code>range</code> arm inside <code>value_node</code>
<code>inherit</code>	YES	<code>css_wide</code> , already in <code>stdlib</code>
<code>%x @y</code>	YES	<code>("%" "@") \$name:ident</code> — verified, refuses <code>plain</code>
<code>&self</code>	YES	<code>"&" \$rest:balanced(';')?</code> — verified, refuses <code>#ff0000</code>
<code>• e •</code>	NO	the lexer has no backtick token

Six of seven are expressible. The seventh is blocked one layer down: a hole is a `PARSER` node (`HTML_HOLE`), and the lexer emits no token for a backtick — documented at `src/syntax/cst/parser.rs:3735` as FUP-046, where it also causes brace miscounting inside template literals.

A capture-type grammar consumes `TOKENS`. No token, no grammar. So the six-sigil check can shrink to a one-sigil check, and closing FUP-046 would remove even that.